

Advanced Image Processing Methods

Homework 6 Report

Peter Dobosi

May 11, 2026

Goal

Compare five pretrained YOLO26 sizes (**n/s/m/l/x**) on the African Wildlife dataset using *inference only*—no training. For every model size, run **predict** on test images to inspect detections qualitatively, then run **val** on the official test split to obtain mAP at three IoU thresholds and wall-clock runtime, and contrast the trade-offs.

Method

1. **Data & models.** The African Wildlife dataset has four classes (**buffalo**, **elephant**, **rhino**, **zebra**) split into 1052/225/227 train/val/test images; only the test split is touched. The five `yolo26{n,s,m,l,x}.pt` weight files supplied via Moodle are loaded as `ultralytics.YOLO` models without modification.
2. **Inference (predict).** A fixed, seeded sample of 16 test images (the same set across all five models) is passed to `model.predict`. Per-image annotated frames with bounding boxes, class labels and confidence scores are rendered through Ultralytics' `Results.plot()` and saved as PNGs. The 16 frames are stitched into a 4×4 montage per model (Figure 1).
3. **Validation (val).** Each model is evaluated on the full 227-image test split via `model.val` with `split="test"`, recording `mAP@0.5`, `mAP@0.75`, `mAP@0.5:0.95`, and the wall-clock runtime measured around the call with `time.perf_counter()`. Ultralytics also writes per-class P/R/F1/PR curves and a confusion matrix into the per-model run directory.
4. **Environment.** A single shared virtualenv on Python 3.14.4 with `ultralytics 8.4.48`, `torch 2.11.0+cpu` and `torchvision 0.26.0+cpu`. All inference and validation runs were executed on CPU (no local GPU).

Results

Model	mAP@0.5	mAP@0.75	mAP@0.5:0.95	Runtime (s)
YOLO26-n	0.962	0.918	0.840	13.95
YOLO26-s	0.974	0.912	0.841	21.62
YOLO26-m	0.968	0.912	0.830	56.50
YOLO26-l	0.973	0.913	0.854	73.38
YOLO26-x	0.974	0.927	0.847	154.02

Table 1: Test-split mAP and wall-clock runtime for the full 227-image **val** pass on CPU. Per-image inference times reported by Ultralytics scale similarly: 32 ms (**n**), 86 ms (**s**), 240 ms (**m**), 314 ms (**l**), 668 ms (**x**).

All five models reach ≥ 0.96 mAP@0.5 on this dataset; the largest spread across sizes is only 0.012 at this threshold and 0.024 at the much stricter mAP@0.5:0.95.

Analysis

How does performance change with model size? The mAP curves are essentially flat. From **n** to **x**, mAP@0.5 ranges in $[0.962, 0.974]$, mAP@0.75 in $[0.912, 0.927]$, and mAP@0.5:0.95 in $[0.830, 0.854]$. The progression is also non-monotonic: **m** is slightly worse than **s** on every metric, and **x** only narrowly tops **l**. With four well-separated, large-instance classes and only 227 test images, this benchmark effectively saturates already at the smallest model, and the residual differences sit within the noise of independently pretrained checkpoints on a small test set.

Is there a point where a larger model no longer helps? Yes—right after **s**. The **n**→**s** step is the only meaningful gain (+0.012 mAP@0.5) and costs a $\sim 2.7\times$ slowdown. Every subsequent step (**s**→**m**, **m**→**l**, **l**→**x**) costs another ~ 1.5 – $2\times$ in inference time but produces sub-percentage-point fluctuations in either direction. The plateau is most visible in mAP@0.5:0.95, where **l** (0.854) and **x** (0.847) are statistically indistinguishable from **n**/**s** (0.840/0.841).

Best accuracy/speed compromise? YOLO26-**n** is the most defensible pick: 32 ms per image (5–21 $\times$ faster than the larger sizes), only 1.2 percentage points behind the best mAP@0.5 and 0.7 pp behind the best mAP@0.5:0.95. If a slightly higher mAP@0.5 is preferred over peak throughput, YOLO26-**s** (0.974 mAP@0.5 at 86 ms/img) is the runner-up. The larger sizes are difficult to justify on this dataset.

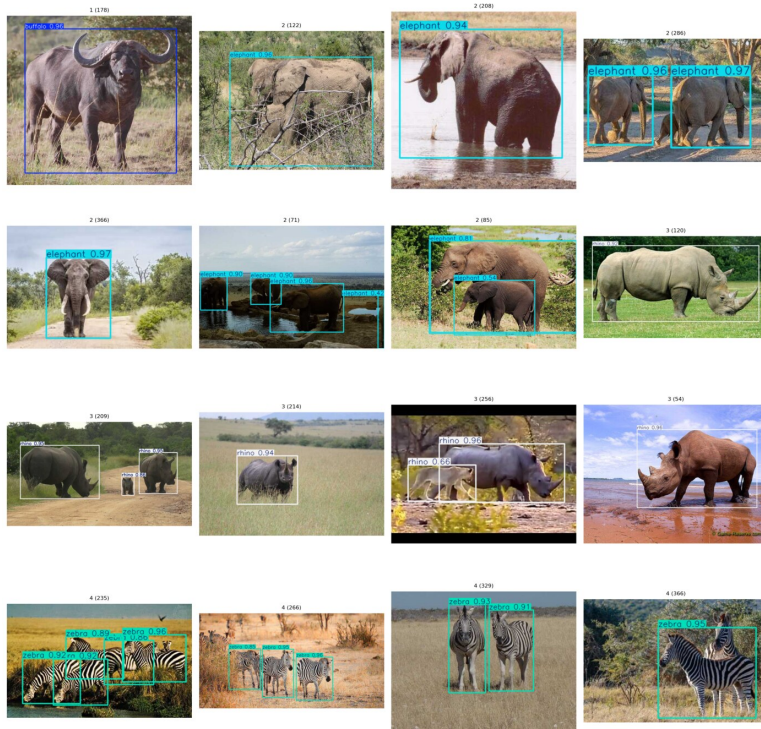


Figure 1: 4×4 montage of YOLO26-**x** detections on the seeded test sample. All four classes are correctly localised and labelled with high confidence; the qualitative output is essentially indistinguishable from the smaller models, consistent with the saturation observed in Table 1.